

Dia C++

Disciplina:
Técnicas de Programação

Turma: S73

Arthur Ferrari Ulbrich e Heitor Garcia Piovan

Tabela de Requisitos Funcionais

1	Apresentar graficamente menu de opções aos usuários do Jogo, no qual pode se escolher fases, escolher ver colocação (<i>ranking</i>) de jogadores e escolher demais opções pertinentes (previstas nos demais requisitos).	REALIZADO	Classe Menu e seu respectivo objeto, com suporte da SFML.	6	Ter três tipos de obstáculos, cada qual com sua representação gráfica, sendo que ao menos um causa dano em jogador se colidirem.	REALIZADO	Pacote Obstáculos, classes Plataforma, Arame Farpado (causa dano ao colidir) e Fosso
2	Permitir um ou dois jogadores com representação gráfica aos usuários do Jogo, sendo que no último caso é para que os dois joguem de maneira concomitante.	REALIZADO	Classe Jogador cujos objetos são agregados em jogo.	7	Ter em cada fase ao menos dois tipos de obstáculos (<u>um deles exclusivo nela</u>) com número aleatório (<u>definindo um máximo</u>) de instâncias	REALIZADO	Classes: Fase, Praia, Trincheira do pacote Fases.
3	Disponibilizar ao menos duas fases <u>distintas</u> que podem ser jogadas sequencialmente ou selecionadas, via menu, nas quais jogadores tentam neutralizar inimigos por meio de algum artifício e vice-versa.	REALIZADO	Classes Fase, Praia, e Trincheira, no pacote Fases, classe Menu, classes do namespace Inimigos e classe Jogador no pacote Personagens		(i.e., objetos), sendo pelo menos 3 instâncias por tipo.		
				8	Ter em cada fase um cenário de jogo constituído por obstáculos, sendo que parte deles devem ser plataformas ou similares, sobre as quais pode haver inimigos e podem subir jogadores. Em cada fase, só poder ter um tipo coincidente de inimigo e um tipo coincidente de obstáculo (que é a plataforma) em relação às demais fases.	REALIZADO	Pacote Obstáculos, classe Plataforma (obstáculo coincidente) e pacote Personagens, classe Melee (Inimigo coincidente), Pacote Fases: classes Fase, Praia e Trincheira.
4	Ter pelo menos três tipos distintos de inimigos, cada qual com sua representação gráfica, sendo que ao menos um deles deve poder lançar projéteis contra o(s) jogador(es) e um dos inimigos deve ser um 'chefão'.	REALIZADO	Pacote Inimigos, Classes Melee, Atirador e Tanque, sendo Atirador e Tanque lançadores de Projéteis.	9	Gerenciar colisões entre jogador para com inimigos e seus projéteis, bem como entre jogador para com obstáculos. Ainda, todos eles devem sofrer o efeito de alguma 'gravidade' no âmbito deste jogo de plataforma vertical e 2D.	REALIZADO	Classe Gerenciador de Colisões e seu respectivo objeto, com suporte da SFML, gravidade atualizada nas próprias entidades
5	Ter a cada fase ao menos dois tipos de inimigos (<u>um deles exclusivo nela</u>) com número aleatório de instâncias, podendo ser várias instâncias (<u>definindo um máximo</u>) e sendo pelo menos 3 instâncias <u>para cada tipo que estiver na fase</u> .	REALIZADO	Classes: Fase, Praia, Trincheira do pacote Fases.	10	Permitir: (1) salvar nome do usuário, manter/salvar pontuação (incrementada via neutralização de inimigos) do jogador controlado pelo usuário e gerar lista de pontuação (<i>ranking</i>). E (2) Pausar e Salvar/Recuperar Jogada.	REALIZADO	Classes Menu, Eventos, Dia e Classes do namespace Entidades.

Total de requisitos funcionais apropriadamente realizados.	100%
--	------

Tabela de Requisitos Conceituais

1	Elementares:		
1.1 &	- Classes, objetos. & Atributos (privados), variáveis e constantes. Métodos (com e sem retorno).	Sim	- Todos .hpp e .cpp, como nas classes nos <i>namespaces</i> Fases e Entidades. - Classes, Objetos, Atributos e Métodos foram utilizados porque são conceitos elementares na orientação a objetos.
1.2 &	- Métodos (com retorno <i>const</i> e parâmetro <i>const</i>). - Construtores (sem/com parâmetros) e destrutores	Sim	- Na maioria dos .hpp e .cpp, como nas classes nos <i>namespaces</i> Personagens e Fases. - A constância pertinente evita mudanças equivocadas, construtores são mandatórios para inicializar atributos e destrutores pertinentes para finalizações como desalocações.
1.3	- Classe Principal.	Sim	- Main.cpp & Principal.hpp/.cpp - Uma classe Principal é mais 'purista' em termos de OO.
1.4	- Divisão em .h e .cpp.	Sim	- No desenvolvimento como um todo, como nas classes nos <i>namespaces</i> Personagens e Fases.. - Permite organizar as classes e afins que compõem o sistema.
2	Relações de:		
2.1	- Associação direcional. & Associação bidirecional.	Sim	- Em vários dos .hpp e .cpp, como nas classes nos <i>namespaces</i> Gerenciadores e Personagens. - Associação direcional é fundamental para organizar e esclarecer dependências entre classes. - Associação bidirecional é fundamental para representar relações mútuas entre classes, onde ambas dependem uma da outra.
2.2 &	- Agregação via associação. - Agregação propriamente dita.	Sim	- Composição: vários .hpp e .cpp, como nas classes Dia, Fase, Praia e Trincheira. O objeto faz parte de outro, mas ainda pode existir por conta própria, o que ajuda a manter tudo mais organizado e reutilizável. - Agregação fraca: Na classe Fase. Um objeto se refere a outro, mas não controla sua existência, dando mais flexibilidade e permitindo que ele seja usado em outros lugares.
2.3 &	- Herança elementar. - Herança em vários níveis.	Sim	- Elementar: Em alguns dos .hpp e .cpp, como nas classes no namespace Fases. - Vários Níveis: Em alguns dos .hpp e .cpp, como nas classes no namespace Entidades. - Reutilização de código.
2.4	- Herança múltipla.	Não	Conceito não utilizado.

3	Ponteiros, generalizações e exceções		
3.1	- Operador <i>this</i> para fins de relacionamento bidirecional	Sim	- Em dia.cpp e menu.cpp - Usar o <i>this</i> permite passar o próprio objeto como referência para o construtor de outro, criando uma ligação direta entre eles.
3.2	- Alocação de memória (<i>new</i> & <i>delete</i>).	Sim	- Em alguns .cpp como nas classes Jogador, Atirador e Tanque. - <i>new</i> e <i>delete</i> permitem alocação dinâmica e controle manual de memória em C++.
3.3	- Gabaritos/ <i>Templates</i> criados/adaptados pelos autores para Listas.	Sim	- Classes Lista (adaptada), Iterator(criada) e IteratorLista(criada). - Gabaritos/templates adaptados para listas facilitam a criação e organização de estruturas de dados de forma eficiente e reutilizável.
3.4	- Uso de Tratamento de Exceções (<i>try catch</i>).	Sim	- No arquivo Dia.cpp. - O uso de <i>try catch</i> permite capturar e tratar erros de forma controlada, garantindo a continuidade do programa e evitando falhas inesperadas.
4	Sobrecarga de:		
4.1	- Construtoras e Métodos.	Sim	- Em Fase e Jogador (Construtoras), Em Fase (Método) - A sobrecarga de construtores e métodos permite criar múltiplas versões com diferentes parâmetros, oferecendo flexibilidade, reutilização e legibilidade no código.
4.2	- Operadores (2 tipos de operadores pelo menos).	Sim	- Foi usado o operador--() na classe Personagem para decrementar a vida (--), e o operador() na estrutura ComparadorJogador, na classe Jogador, para comparar pontuação entre ponteiros de jogadores. - A sobrecarga de operadores permite personalizar operadores para tipos definidos pelo usuário, tornando o código mais intuitivo.
---	Persistência de Objetos (via arquivo de texto ou binário)		
4.3	- Persistência de Objetos.	Sim	- Em Fase e namespace Entidades, via JSON. - Salvamento de Entidades para manter a jogada.
4.4	- Persistência de Relacionamento de Objetos.	Não	Conceito não utilizado.

Tabela de Requisitos Conceituais

5	Virtualidade:		
5.1	- Métodos Virtuais Usuais.	Sim	-Em algumas classes, como Personagem e Entidade. -Métodos virtuais permitem polimorfismo, com comportamentos específicos em subclasses.
5.2	- Polimorfismo.	Sim	-Em várias classes, como Personagem e Entidade -Polimorfismo permite que um objeto se comporte de maneiras diferentes dependendo do contexto.
5.3	- Métodos Virtuais Puros / Classes Abstratas.	Sim	-Em várias classes, como Ente e Fase -Métodos virtuais puros e classes abstratas garantem implementação obrigatória em subclasses, promovendo flexibilidade.
5.4	- Coesão/Desacoplamento efetiva e intensa com o apoio de padrões de projeto (mais de 5 padrões).	Não	Foram utilizados apenas 2 padrões de projeto.
6	Organizadores e Estáticos		
6.1	- Espaço de Nomes (<i>Namespace</i>) criado pelos autores.	Sim	-Namespace Inimigos -Espaços de nomes organizam o código, evitando conflitos de nomes e melhorando a legibilidade e manutenção.
6.2	- Classes aninhadas (<i>Nested</i>) criada pelos autores.	Sim	-Classe IteratorLista aninhado com Iterator no namespace Listas. -Proporcionam melhor encapsulamento e organização, além de contribuir para a legibilidade e a manutenção.
6.3	- Atributos estáticos e métodos estáticos.	Sim	-Em algumas classes como Ente, Jogador e classes do pacote Gerenciadores. -São compartilhados com todas as instâncias de classe.
6.4	- Uso extensivo de constante (<i>const</i>) parâmetro, retorno, método...	Sim	-Na maioria dos .hpp e .cpp. -Evitar erros e alterações indesejadas, promovendo segurança.

7	Standard Template Library (STL) e String OO		
7.1	- A classe Pré-definida <i>String</i> ou equivalente. & - <i>Vector</i> e/ou <i>List</i> da <i>STL</i> (p/ objetos ou ponteiros de objetos de classes definidos pelos autores)	Sim	-Na classe Jogador (vector e string). - String : facilita a manipulação de sequências de caracteres. - Vector : oferecem armazenamento dinâmico e gerenciamento automático de memória para objetos.
7.2	- Pilha, Fila, Bifila, Fila de Prioridade, Conjunto, Multi-Conjunto, Mapa OU Multi-Mapa.	Sim	-Classe Gerenciador Gráfico e Classe Dia. -Mapa: Permite armazenar e acessar pares chave-valor de forma eficiente e ordenada. -Multi-Conjunto: Permite armazenar de forma ordenada elementos, podendo haver repetição de “chave”
---	Programação concorrente		
7.3	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos, utilizando Posix, C-Run-Time OU Win32API ou afins.	Não	NÃO realizado no tempo previsto
7.4	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos com uso de Mutex, Semáforos, OU Troca de mensagens.	Não	NÃO realizado no tempo previsto

Tabela de Requisitos Conceituais

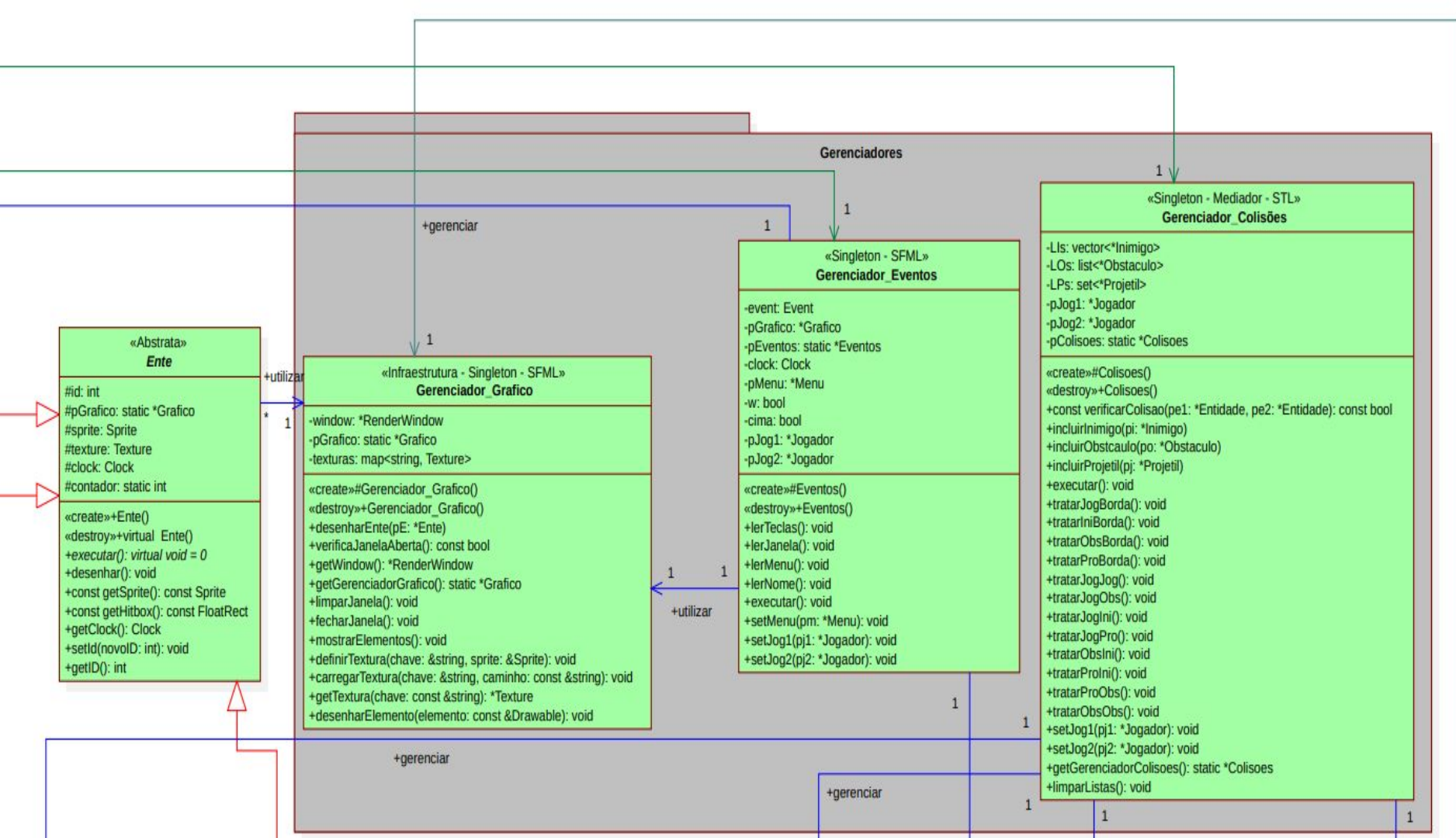
8	Biblioteca Gráfica / Visual		
8.1 &	- Funcionalidades Elementares. - Funcionalidades Avançadas como: tratamento de colisões, duplo <i>buffer</i> ou outros.	Sim	-Tratamento de Colisões (em Gerenciador de Colisões). -Leitura de Eventos (em Gerenciador de Eventos). -Renderização de janela e desenhar elementos em tela.
8.2 O U	- Programação orientada e evento efetiva (com gerenciador apropriado de eventos inclusive, via padrão de projeto <i>Observer</i>) em algum ambiente gráfico. - <i>RAD – Rapid Application Development</i> (Objetos gráficos como formulários, botões etc).	Não	Gerenciador de Eventos SEM Observer
---	Interdisciplinaridades via utilização de Conceitos de Matemática Contínua e/ou Física.		
8.3	- Ensino Médio Efetivamente.	Sim	-Gravidade com aceleração (movimento uniformemente acelerado). -Utilizado para fazer com que o jogo tenha gravidade funcional na jogabilidade.
8.4	- Ensino Superior Efetivamente.	Não	-Nenhum conceito utilizado.

9	Engenharia de Software		
9.1	- Compreensão, melhoria e rastreabilidade de cumprimento de requisitos.	Sim	-Os requisitos foram cumpridos por meio da implementação conforme especificações, validação por testes e rastreabilidade dos requisitos. -Garantir compreensão, melhoria e rastreabilidade dos requisitos assegura que o produto atenda às necessidades e facilita a verificação de seu cumprimento.
9.2	- Diagrama de Classes em <i>UML</i> .	Sim	-Respectivo arquivo do Diagrama de Classes -O Diagrama de Classes em UML representa a estrutura do sistema, facilitando o entendimento e a comunicação entre a equipe.
9.3	- Uso efetivo e intensivo de padrões de projeto <i>GOF</i> , <i>i.e.</i> , mais de 5 padrões.	Não	Foram utilizados apenas 2 padrões de projeto.
9.4	- Testes à luz da Tabela de Requisitos e do Diagrama de Classes.	Sim	-Garante que o sistema atenda aos requisitos especificados e que as interações entre as classes sejam corretamente implementadas e validadas.

Tabela de Requisitos Conceituais

10	Execução de Projeto			10.	- Reuniões com monitor da disciplina para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	Sim	-5 reuniões do PETECO em que ambos os integrantes da dupla participaram -Datas e horários: 1- 29/11/2024, 13:00 até 14:40 2- 05/12/2024, 15:50 até 16:40 3- 06/12/2024, 13:00 até 14:40 4- 12/12/2024, 17:00 até 18:20 5- 13/12/2024, 13:00 até 14:40 - Essencial para se adequar com a biblioteca gráfica, forma de projeto e cumprimento de requisitos.
10.1	- Controle de versão de modelos e códigos automatizado (via github). - Uso de alguma forma de cópia de segurança (i.e., <i>backup</i>).	Sim	-Google Drive para backup https://github.com/HeitorGPiovan/Jogo	3			
10.2	- Reuniões com o professor para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	Sim	-2 reuniões, a primeira no dia 11/12/2024 e a segunda no dia 18/12/2024, ambas no horário de 15:30 -Emails enviados nos dias acima para o Professor, constando as informações requisitadas. - Garantiu que o projeto estivesse no caminho correto, lapidando e aperfeiçoando o desenvolvimento.	10.4	- Escrita do trabalho e feita da apresentação - Revisão do trabalho escrito de outra equipe e vice-versa.	Sim	- Ambos contribuíram significativamente tanto na escrita do trabalho quanto na elaboração da apresentação de uma maneira geral. -Dupla responsável pela revisão do trabalho: Lucas Schaefer e Matheus Valente
&							

Total de conceitos apropriadamente utilizados.	80%
---	------------



Entidades

«Abstrata»

Entidade

#x: int
#y: int
#tipo: int
#direcao: int
#velocidadeVertical: float
#gravidade: const float

«create»+Entidade()
«destroy»+virtual Entidade()
+executar(): virtual void = 0
+salvar(): virtual void = 0
+salvarDataBuffer(&j: json): void
+setY(PosY: const int): void
+setX(PosX: const int): void
+setDir(dir: const int): void
+const getY(): const int
+const getX(): const int
+const getDir(): const int
+atualizarGravidade(): virtual void
+const getVel(): const float
+const getTipo(): const int
+setVel(vel: const float): void
+setTipo(NovoTipo: const int): void
+exportarParaArquivo(&nomeArquivo: const string, &dados_json: const json): void

+gerenciar

realiza

+gerenciar

«Entidade»

Projetoil

-ativo: bool
-dano: int
-delnimigo: bool

«create»+Projetoil(dir: int, x: int, y: int, dmg: int)
«create»+Projetoil()
«destroy»+Projetoil()
+executar(): void
+transitar(): void
+acertar(p: *Jogador): void
+const getAtivo(): const bool
+setAtivo(atv: const bool): void
+salvar(): void
+setDano(dmg: int): void
+setDelnimigo(dir: bool): void
+const getDelnimigo(): const bool

+disparar

0..1

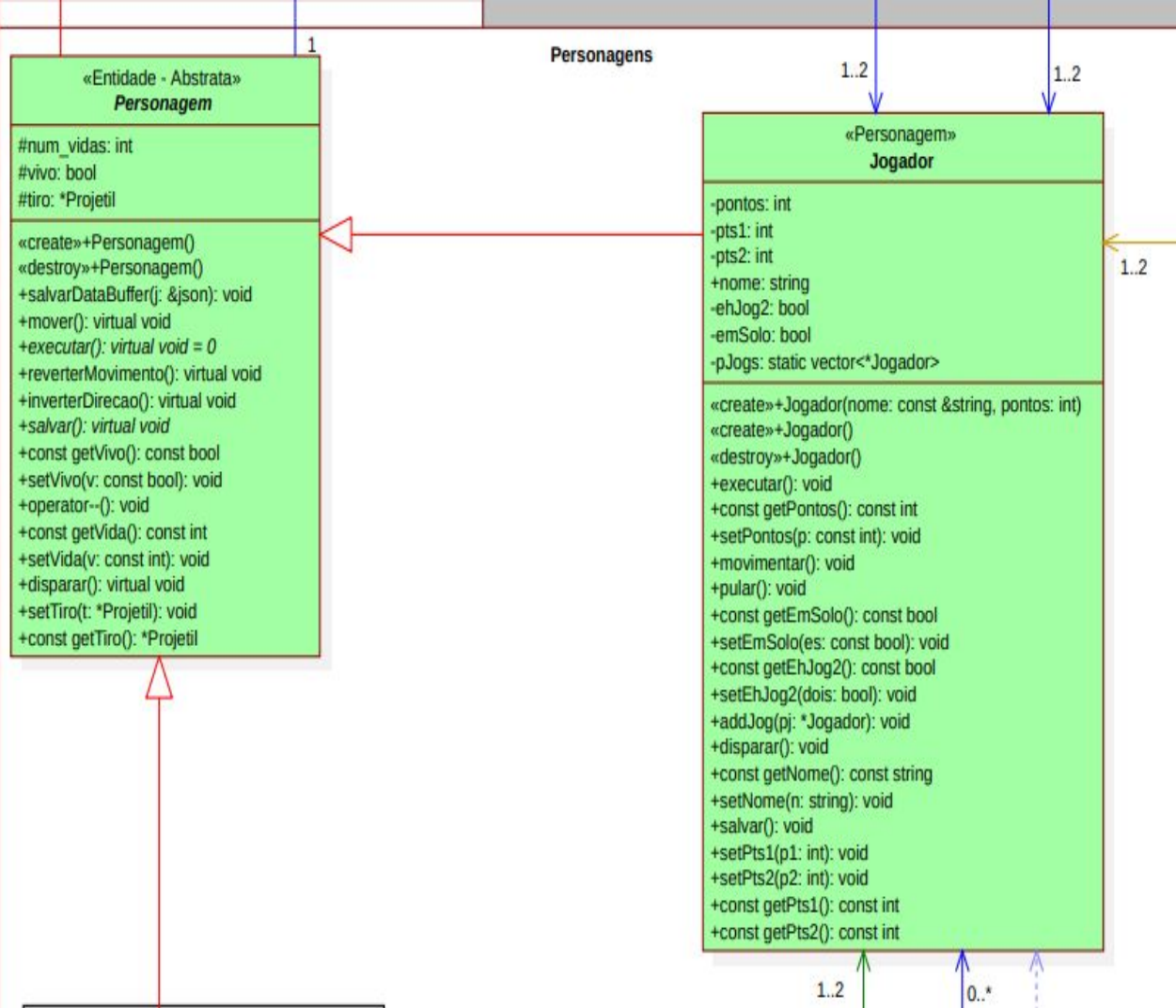
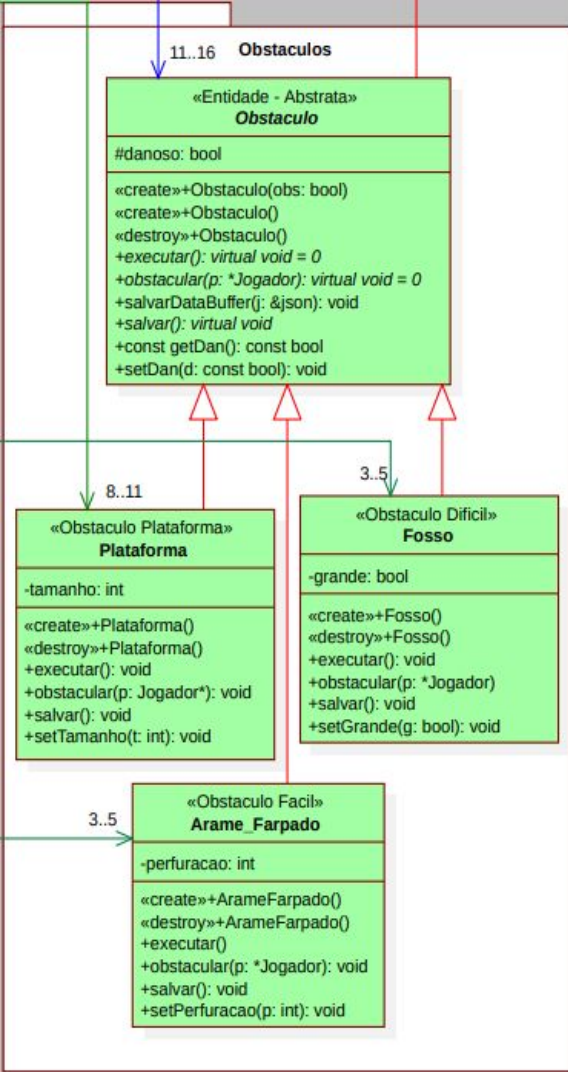
1

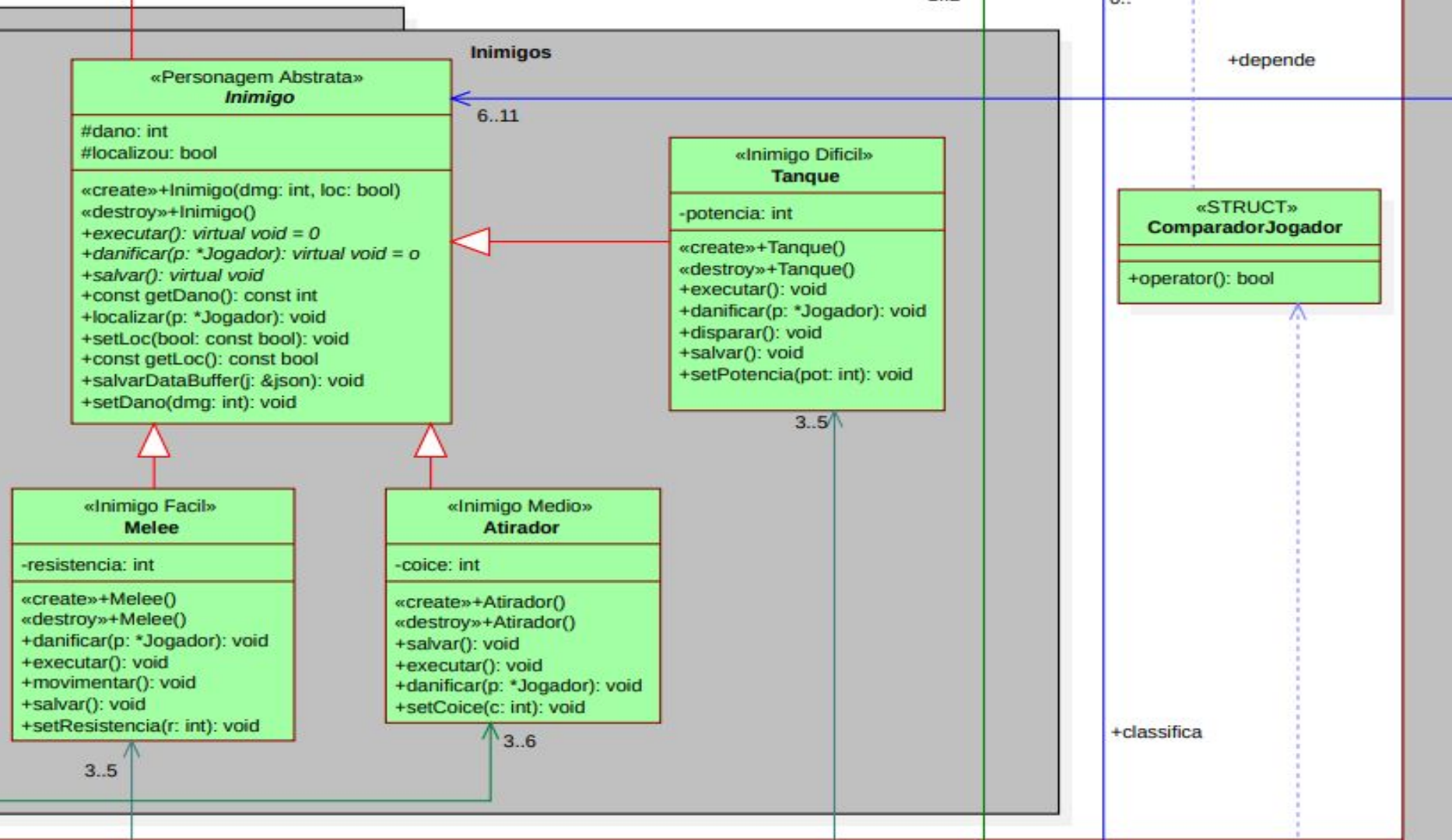
+gerenciar

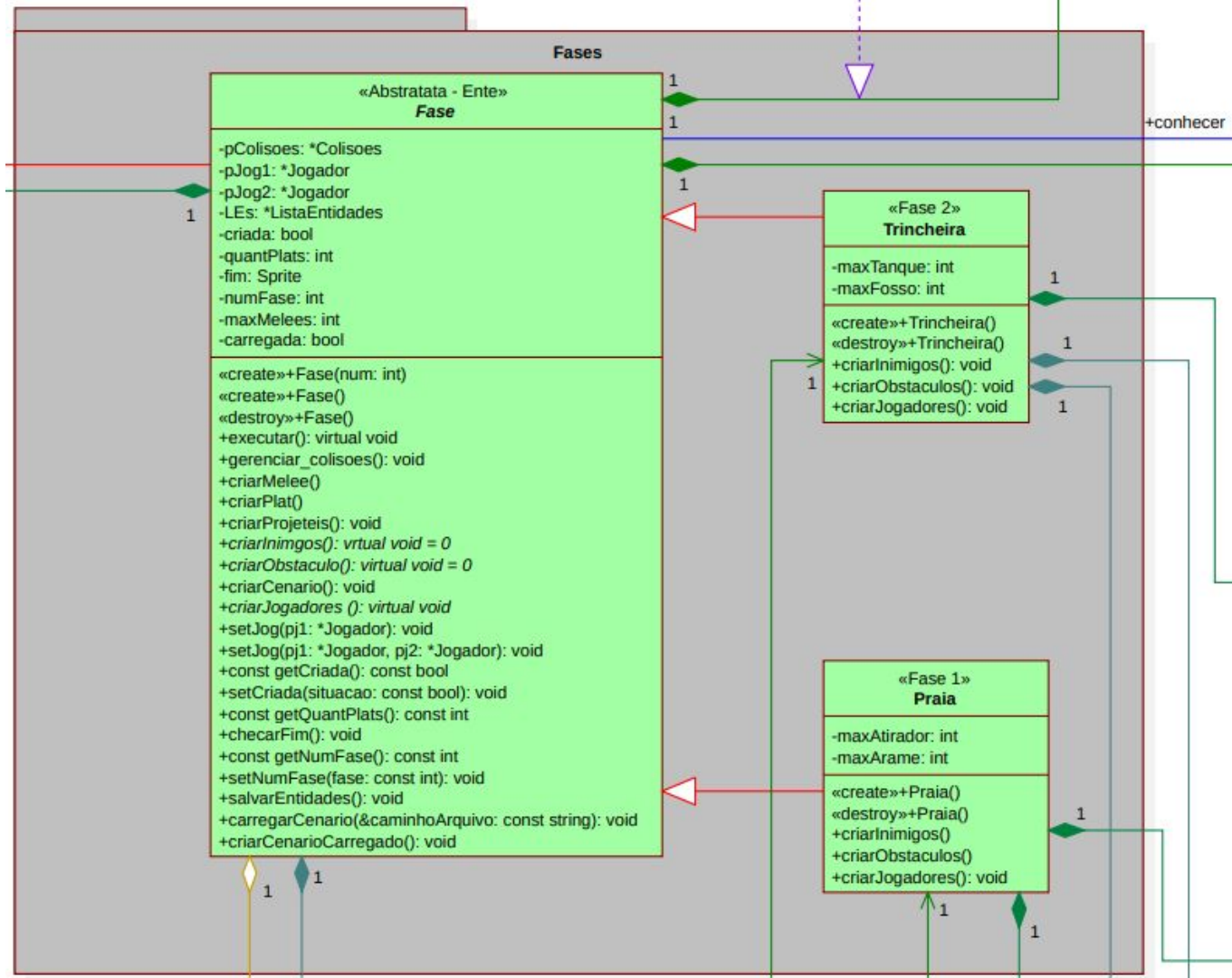
4..16

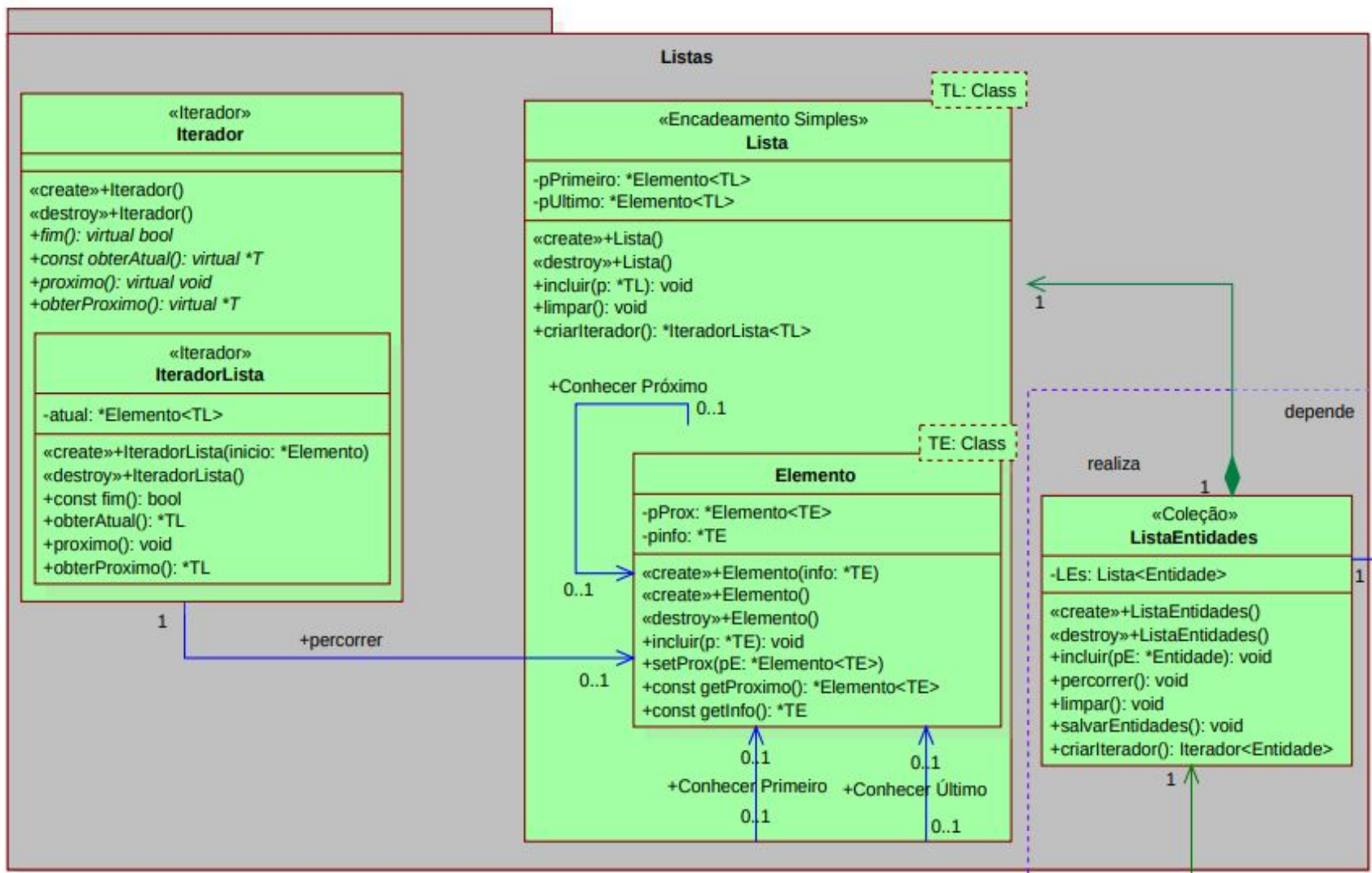
22..45

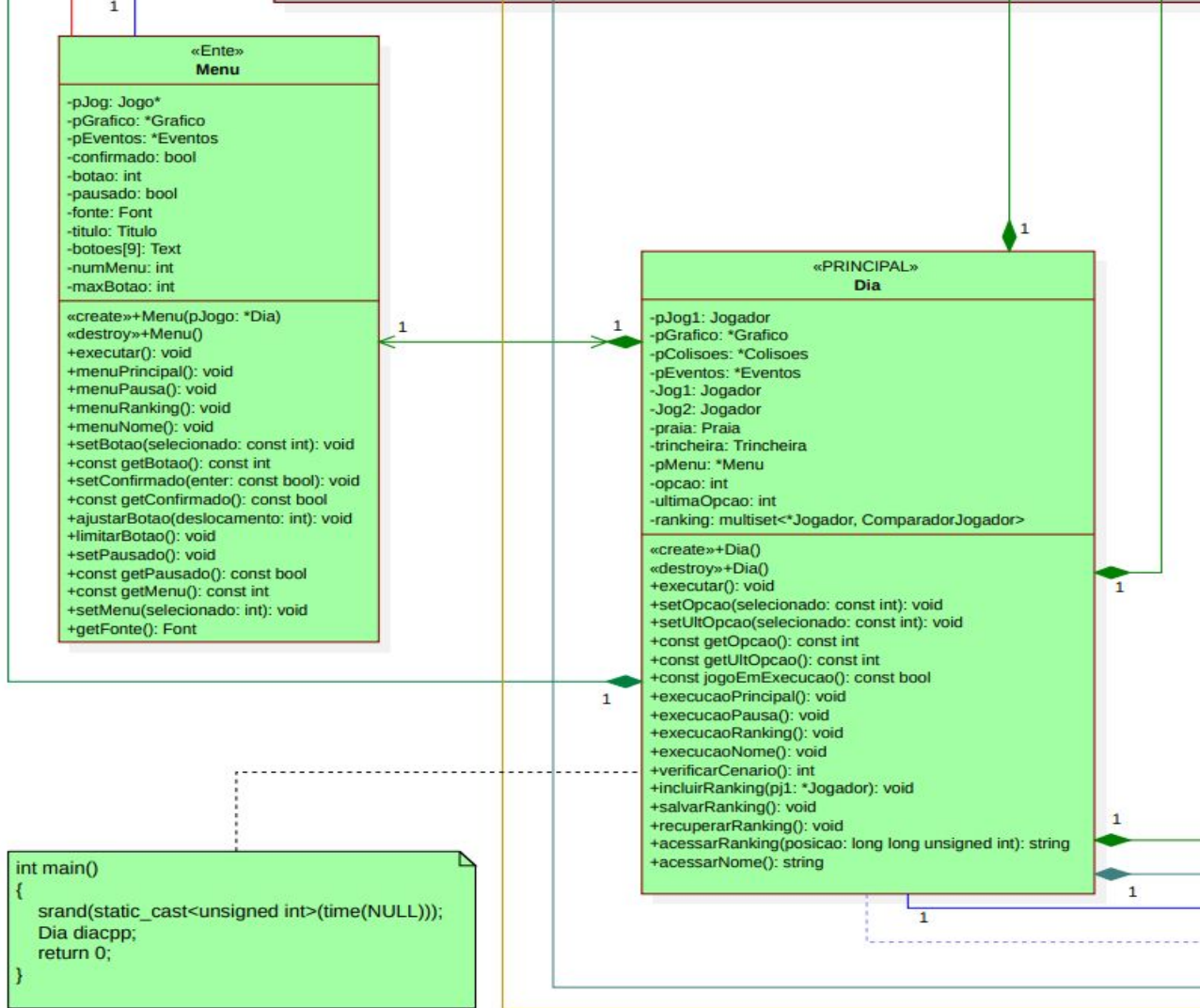
22..45











MENU PRINCIPAL

Fase 1 - Singleplayer

Fase 1 - Multiplayer

Fase 2 - Singleplayer

Fase 2 - Multiplayer

Carregar

Ranking

RANKING

C - 90

A - 75

B - 70

-

-

-

-

-

Sair

PAUSADO

Continuar

Salvar

Cadastrar Nome

Sair

CADASTRAR NOME

Arthur

Sair





Conclusão Geral dos Resultados Alcançados

O desenvolvimento do jogo em C++ com POO e SFML permitiu a aplicação de conceitos essenciais, resultando no cumprimento de 100% dos requisitos funcionais e na implementação de 80% dos conceitos previstos. A modelagem em UML facilitou a organização do projeto, garantindo um design estruturado e eficiente. O método simplificado de engenharia de software possibilitou um desenvolvimento controlado e produtivo. O projeto consolidou conhecimentos vistos ao longo da disciplina e reforçou boas práticas de programação, projeto e trabalho em equipe, garantindo um aprendizado significativo.

Total de requisitos funcionais apropriadamente realizados.		100%
Total de conceitos apropriadamente utilizados.	80%	